HireSense AI

Product Requirements Document (PRD)

---

1. Product Overview

Product Name: HireSense AI
Product Type: Web-based AI recruitment assistant.

Core Problem:
• Applicants do not know whether their resumes match job requirements.
• Users are unaware of their skill gaps.
• Many users spam job applications inefficiently.
• ATS rejection rates are high.

Recruiters also face major challenges:
• Resume screening is time-consuming.
• Candidate ranking is inefficient.
• Manual CV review wastes time.

Product Goal:
Build a platform that: • analyzes resumes, • compares resumes with job descriptions, • generates compatibility scores, • identifies missing skills, • helps recruiters rank candidates automatically.

2. Target Users

Primary Users:

Job Seekers
• Students
• Fresh graduates
• Junior to mid-level professionals

Recruiters / HR
• Startup recruiters
• Internal HR teams
• Freelance recruiters

3. Core Value Proposition

For Candidates:
• Understand job compatibility
• Identify missing skills
• Improve ATS compatibility

For Recruiters:
• Faster candidate screening

• Automatic ranking
• Reduced manual review workload

4. MVP Scope

The MVP must focus only on: **Resume ↔ Job Description Matching**

This is not a full career platform. Avoid overbuilding before the core system is stable.

5. MVP Features

Authentication
• Candidate registration/login
• Recruiter registration/login
• JWT authentication
• Role-based access

Resume Upload
• Upload PDF/DOCX resumes
• File size limitation
• Automatic parsing

Extracted Data
• Skills
• Education
• Experience
• Projects
• Certifications

Job Description Input
• Recruiters can create job postings
• Candidates can paste job descriptions manually

AI Matching Engine Output
• Matching percentage
• Matched skills
• Missing skills
• Semantic relevance score

Resume Feedback
• Weak wording detection
• Missing keyword suggestions
• ATS optimization recommendations

Candidate Dashboard
• Upload history
• Match history
• Saved jobs
• Improvement tracking

Recruiter Dashboard
• Candidate ranking
• Candidate shortlist

• Filtering
• Job management

6. Non-MVP Features

Do NOT build these initially:
• AI interview system
• Video analysis
• Personality prediction
• Emotion detection
• AI avatar
• Blockchain integrations
• Kubernetes
• Complex microservices architecture

These are distractions during the MVP phase.

7. System Architecture

High-Level Architecture:

Next.js Frontend
↓
FastAPI Backend
↓
ML/NLP Service
↓
PostgreSQL + pgvector

8. Technical Requirements

Frontend Stack
• Next.js
• TailwindCSS
• shadcn/ui

Responsibilities: • Authentication UI
• Dashboard UI
• Resume upload UI
• Match visualization

Backend Stack
• FastAPI

Responsibilities: • REST API
• Authentication
• File handling
• Async tasks
• Service orchestration

ML/NLP Stack

• sentence-transformers
• scikit-learn
• spaCy

ML Tasks: • Embedding generation
• Semantic similarity
• Keyword extraction
• Candidate ranking

Database
• PostgreSQL
• pgvector

9. ML Pipeline

Resume Processing Flow
Upload Resume → PDF Parsing → Text Cleaning → Skill Extraction → Embedding Generation → Store in Vector DB

Matching Flow
Job Description → Embedding Generation → Semantic Similarity → Ranking → Match Score

10. API Design

Candidate APIs
POST /upload-resume
GET /resume-analysis/:id
POST /match-job

Recruiter APIs
POST /jobs
GET /jobs/:id/candidates

11. Database Schema

users
id, name, email, password_hash, role, created_at

resumes
id, user_id, file_url, parsed_text, embedding, created_at

jobs
id, recruiter_id, title, description, embedding, created_at

matches
id, resume_id, job_id, score, missing_skills, created_at

12. AI/ML Strategy

Avoid building custom deep learning models too early.

Use: • Pretrained embeddings
• Semantic similarity
• Retrieval-based approaches

Why? • Faster development
• More stable systems
• More production-ready

13. Recommended ML Stack

Embeddings
• BAAI/bge-small-en
• all-MiniLM-L6-v2

Similarity
• Cosine similarity

Parsing Tools
• spaCy
• PyMuPDF
• pdfplumber

14. Security Requirements

Must-have security features:
• Password hashing
• JWT authentication
• Rate limiting
• File validation
• Upload sanitization

File uploads are a major attack surface and must be handled carefully.

15. Logging & Monitoring

Logging
• Request logs
• Inference logs
• Upload logs

Monitoring
• API response time
• Inference latency
• Failure rates

16. Success Metrics

Technical Metrics
• API latency < 2 seconds
• Resume parsing success > 90%

• Acceptable semantic matching quality

Product Metrics
• Repeat usage
• Number of uploaded resumes
• Recruiter engagement

17. Development Roadmap

Phase 1 — Core MVP (2–3 Weeks)
• Authentication
• Resume upload
• Job description input
• Matching engine
• Basic dashboard

Phase 2 — Production Hardening
• Docker
• Logging
• Redis queue
• Improved parsing
• Deployment

Phase 3 — Advanced AI
• Resume rewriting
• AI recommendations
• Recruiter analytics
• Multi-job ranking

18. Biggest Risks

Risk #1: Overengineering too early.
Risk #2: Focusing on UI aesthetics before backend completion.
Risk #3: Building custom ML models too soon.
Risk #4: Not deploying the system.

An ML project without deployment has significantly lower portfolio value.

19. Strategic Advice

What makes this project valuable is NOT: • Fancy animations
• Overdesigned UI
• Unrealistic accuracy claims

What matters: • Clean architecture
• Reliable deployment
• Production-grade APIs
• System reliability
• Real usability

If this project includes: • Dockerized backend

• Deployed frontend
• Vector search
• Proper authentication
• Clean API design
• Recruiter workflow support

Then it already demonstrates strong junior-level ML engineering capability.

20. Execution Plan

STEP 1 — Define Scope Clearly
Candidate: • Register/login
• Upload resume PDF
• Input job description
• Get matching score
• View missing skills

Recruiter: • Create job posting
• View ranked candidates

Do NOT add: • Voice AI
• Chatbots
• Realtime systems
• Notifications
• Mobile apps

STEP 2 — Build Backend FIRST
Focus on: • Authentication
• Upload endpoints
• File storage
• Database connection

Priority endpoints: POST /register
POST /login
POST /upload-resume
POST /jobs
POST /match

STEP 3 — Setup Database
Use: • PostgreSQL
• pgvector

This helps you learn: • Semantic search
• Vector storage
• Embedding similarity

STEP 4 — Resume Parsing
Extract text from: • PDF
• DOCX

Use: • PyMuPDF
• pdfplumber

Example output: {
  "skills": [],

```
  "experience": "",
  "education": ""
}
```

STEP 5 — Embeddings
Use pretrained models: • sentence-transformers
• all-MiniLM-L6-v2

Flow: Resume Text → Embedding → Store Vector

STEP 6 — Matching Engine
Use cosine similarity first.
Do not overcomplicate the system with custom deep learning models.

STEP 7 — Frontend
Only build the UI after: • API contracts are stable
• Data structures are clear
• System flow is validated